# API Gateway Tabanlı Mikroservis Mimarisinin Tasarımı, Uygulanması ve Performans Analizi

# Design, Implementation and Performance Analysis of an API Gateway-Based Microservices Architecture

1. Eyüp CANPOLAT
*221307011*
*Bilişim Sis. Müh.*
*22130701@kocaeli.edu.tr*

2. Yağız BİLGİLİ
*231307120*
*Bilişim Sis. Müh.*
*231307120@kocaeli.edu.tr*

***Özetçe*— Günümüzde yazılım sistemlerinin artan karmaşıklığı ve ölçeklenebilirlik gereksinimleri, geleneksel monolitik mimarilerin yetersiz kalmasına neden olmuştur. Bu bağlamda, mikroservis mimarisi; bağımsız geliştirilebilir, dağıtılabilir ve ölçeklenebilir servisler sunarak modern yazılım geliştirme süreçlerinde önemli bir çözüm haline gelmiştir. Mikroservis yaklaşımı, sistem bileşenlerini küçük ve yönetilebilir parçalara ayırarak hem geliştirme sürecini hızlandırmakta hem de bakım maliyetlerini azaltmaktadır.**

**Bu çalışmada, mikroservis mimarisi temel alınarak geliştirilen bir sistemin uçtan uca tasarımı ve gerçekleştirilmesi ele alınmıştır. Sistem, istemci isteklerini karşılayan merkezi bir Dispatcher (API Gateway) bileşeni ve bu bileşen tarafından yönlendirilen bağımsız mikroservislerden oluşmaktadır. Dispatcher, sistemin tek giriş noktası olarak görev yapmakta ve tüm yönlendirme, kimlik doğrulama ve yetkilendirme işlemlerini yönetmektedir. Bu sayede sistem güvenliği artırılırken, servisler arası iletişim daha kontrollü hale getirilmiştir.**

**Geliştirilen sistemde, yazılım kalitesini artırmak amacıyla Test-Driven Development (TDD) yaklaşımı benimsenmiş ve özellikle Dispatcher bileşeni bu metodoloji ile geliştirilmiştir. Ayrıca, her mikroservis için bağımsız veri tabanı kullanılarak veri izolasyonu sağlanmış ve MongoDB tercih edilmiştir. Sistemin performansı ise k6 aracı ile gerçekleştirilen yük testleri ile analiz edilmiştir. Bu çalışma, modern yazılım geliştirme tekniklerinin bir arada kullanıldığı bütünleşik bir çözüm sunmayı amaçlamaktadır.**

***Anahtar Kelimeler — Mikroservis Mimarisi, API Gateway, Dispatcher, Go (Golang), MongoDB, Test Odaklı Geliştirme (TDD), RESTful API, Richardson Olgunluk Modeli, k6 Yük Testi, Docker, Dağıtık Sistemler.***

***Abstract*—This study presents the design and implementation of a scalable and secure microservices-based system developed using the Go programming language. The system architecture is built upon a centralized Dispatcher (API Gateway) that manages all incoming requests and routes them to independent microservices. Each service operates with its own isolated data storage using MongoDB, ensuring loose coupling and high cohesion across the system.**

**The Dispatcher component is developed following the Test-Driven Development (TDD) methodology to improve code reliability and maintainability. Authorization and authentication mechanisms are centrally handled, while internal services are isolated from external access, enhancing overall system security. The architecture adheres to RESTful principles and follows the Richardson Maturity Model Level 2 standards for API design.**

**Performance and scalability of the system are evaluated using the k6 load testing tool under various concurrent request scenarios. The results demonstrate that the system can efficiently handle high traffic loads with low latency and minimal error rates. This project highlights the practical integration of modern software engineering practices such as microservices architecture, containerization, TDD, and performance testing.**

***Keywords — Microservices Architecture, API Gateway, Dispatcher, Go (Golang), MongoDB, Test-Driven Development (TDD), RESTful API, Richardson Maturity Model, k6 Load Testing, Docker, Distributed Systems.***

## I. INTRODUCTİON (GİRİŞ)

Günümüzde yazılım sistemlerinin artan kullanıcı yükü, veri hacmi ve esneklik gereksinimleri, geleneksel monolitik mimarilerin sınırlarını zorlamaktadır. Tek parça halinde geliştirilen bu sistemler, ölçeklenebilirlik, bakım kolaylığı ve hata yönetimi açısından çeşitli dezavantajlar barındırmaktadır. Bu bağlamda, modern yazılım geliştirme yaklaşımları arasında öne çıkan mikroservis mimarisi, sistem bileşenlerini bağımsız ve birbirinden izole servisler halinde tasarlayarak daha esnek, sürdürülebilir ve ölçeklenebilir çözümler sunmaktadır.

Bu çalışmada, mikroservis mimarisi temel alınarak geliştirilen bir yazılım sisteminin uçtan uca tasarımı ve gerçekleştirilmesi ele alınmaktadır. Geliştirilen sistemde, tüm istemci isteklerini karşılayan merkezi bir Dispatcher (API Gateway) bileşeni yer almakta ve bu bileşen aracılığıyla gelen talepler ilgili mikroservislere yönlendirilmektedir. Dispatcher, sistemin tek giriş noktası olarak görev yapmakta; yönlendirme, kimlik doğrulama, yetkilendirme ve hata yönetimi gibi kritik işlevleri üstlenmektedir. Bu yapı sayesinde hem sistem güvenliği artırılmış hem de servisler arası iletişim kontrol altına alınmıştır.

Sistem kapsamında geliştirilen mikroservisler, birbirinden bağımsız çalışacak şekilde tasarlanmış ve her biri kendi veri

tabanına sahip olacak biçimde yapılandırılmıştır. Veri yönetimi için NoSQL tabanlı bir çözüm olan MongoDB tercih edilerek yüksek performans ve esneklik sağlanmıştır. Ayrıca sistemin geliştirilmesinde Go programlama dili kullanılmış olup, özellikle Dispatcher bileşeni Test-Driven Development (TDD) yaklaşımı ile geliştirilerek yazılım kalitesi artırılmıştır.

Bunun yanı sıra, sistemin performansı ve dayanıklılığı, gerçek dünya senaryolarını simüle eden yük testleri ile analiz edilmiştir. Bu amaçla k6 aracı kullanılarak farklı eş zamanlı kullanıcı yükleri altında sistemin tepki süreleri ve hata oranları ölçülmüştür. Elde edilen bulgular, geliştirilen mimarinin yüksek trafik altında dahi kararlı ve verimli bir şekilde çalışabildiğini göstermektedir. Bu çalışma, modern yazılım geliştirme pratiklerinin bir arada kullanıldığı bütünleşik bir sistem tasarımını ortaya koymayı amaçlamaktadır.

## II. Background & Related Concepts (Kuramsal Arka Plan)

Modern yazılım geliştirme süreçlerinde, sistemlerin ölçeklenebilir, sürdürülebilir ve esnek bir yapıda tasarlanması büyük önem taşımaktadır. Bu gereksinimlerin karşılanabilmesi amacıyla mikroservis mimarisi, RESTful servis tasarımı, Test-Driven Development (TDD) yaklaşımı ve olgun API tasarım modelleri gibi çeşitli kavramlar yaygın olarak kullanılmaktadır. Bu bölümde, geliştirilen sistemin temelini oluşturan bu kavramlar açıklanmaktadır.

### *A. Mikroservis Mimarisi (Microservices Architecture)*

Mikroservis mimarisi, bir yazılım sisteminin küçük, bağımsız ve belirli işlevlere odaklanan servisler halinde tasarlanmasını ifade eder. Her bir mikroservis kendi iş mantığını içerir ve diğer servislerden bağımsız olarak geliştirilebilir, dağıtılabilir ve ölçeklenebilir. Bu yaklaşım, özellikle büyük ve karmaşık sistemlerde modülerliği artırarak bakım süreçlerini kolaylaştırmaktadır.

Bu mimaride servisler arası iletişim genellikle hafif protokoller (örneğin HTTP/REST) ve veri formatları (JSON gibi) kullanılarak gerçekleştirilir. Ayrıca, her mikroservisin kendi veri tabanına sahip olması prensibi (database per service) sayesinde veri izolasyonu sağlanır. Bu durum, sistemin hata toleransını artırırken servisler arası bağımlılığı azaltmaktadır.

### *B. API Gateway (Dispatcher) Yaklaşımı*

Mikroservis mimarisinde istemci ile servisler arasındaki iletişimi yönetmek amacıyla API Gateway (Dispatcher) yapısı kullanılmaktadır. Bu yapı, sistemin tek giriş noktası olarak görev yapar ve gelen tüm istekleri ilgili mikroservislere yönlendirir. Aynı zamanda kimlik doğrulama (authentication), yetkilendirme (authorization), loglama ve hata yönetimi gibi çapraz kesen (cross-cutting) işlemler bu katmanda gerçekleştirilir.

API Gateway kullanımı, istemci tarafının sistemin iç yapısını bilmesini gereksiz kılar ve tüm servislerin merkezi bir noktadan yönetilmesini sağlar. Bu yaklaşım, güvenlik ve kontrol mekanizmalarının daha etkin bir şekilde uygulanmasına olanak tanımaktadır.

### *C. RESTful Servisler ve Richardson Olgunluk Modeli*

REST, dağıtık sistemlerde kaynaklara erişimi standartlaştıran bir mimari yaklaşımdır. RESTful servislerde işlemler, HTTP metodları (GET, POST, PUT, DELETE) kullanılarak gerçekleştirilir ve her kaynak URI ile temsil edilir.

Richardson Olgunluk Modeli (RMM), REST servislerini seviyelere ayırır. Bu projede Seviye 2 benimsenmiş olup, kaynak bazlı endpoint kullanımı ve HTTP metodlarının doğru şekilde uygulanması sağlanmıştır.

### *D. Sınıf (Struct) Yapıları ve Katmanlı Mimari (Clean Architecture)*

Sistemdeki her bir Go mikroservisi "Clean Architecture" prensiplerine sadık kalarak kendi içinde dört ana katmana ayrılmıştır:

1. Model: Veritabanı tablolarını/koleksiyonlarını ve DTO (Data Transfer Object) yapılarını barındıran salt veri nesneleri.
2. Repository: Doğrudan MongoDB sürücüleri ile iletişim kuran veri yönetim katmanı. Veri erişim soyutlaması yapılması, gelecekte MongoDB'den sistem esnekliğini bozmadan PostgreSQL'e gibi bir sisteme geçiş yapılabilmesini kolaylaştırır.
3. Service: Sistemin temel iş mantığını (Business Logic) üzerinde tutan katman. Doğrulamalar, hesaplamalar ve hata fırlatmalar burada yapılır.
4. Handler (Controller): Gelen HTTP isteklerini karşılayan, verileri struct yapılarına çeviren ve sonucu servis katmanından alarak istemciye ulaştıran dış çeper.

### *E. Test-Driven Development (TDD)*

TDD, testlerin koddan önce yazıldığı bir geliştirme yaklaşımıdır. Süreç, Red-Green-Refactor döngüsüne dayanır. Bu yöntem, daha güvenilir ve sürdürülebilir yazılım geliştirilmesini sağlar. Bu projede Dispatcher bileşeni TDD yaklaşımı ile geliştirilmiştir.

### *F. NoSQL Veri Tabanları ve MongoDB*

NoSQL veri tabanları, esnek veri modelleri sunarak dağıtık sistemler için uygun çözümler sağlar. MongoDB, verileri JSON benzeri formatta saklayan belge tabanlı bir NoSQL veri tabanıdır. Bu projede veri izolasyonu ve performans avantajı sağlamak amacıyla tercih edilmiştir.

### *G. Yük Testi ve k6 Aracı*

Yük testi, sistemin farklı kullanıcı yükleri altındaki performansını ölçmek için kullanılır. Bu projede k6 aracı ile eş zamanlı istek senaryoları oluşturulmuş ve sistemin yanıt süreleri ile hata oranları analiz edilmiştir.

## III. SİSTEM MİMARİSİ VE TEORİK ALTYAPI

Bu bölümde geliştirilen sistemin genel mimarisi, kullanılan teorik yaklaşımlar ve bileşenler arası ilişkiler ele alınmaktadır. Sistem, mikroservis mimarisi temel alınarak tasarlanmış olup tüm istemci istekleri merkezi bir API Gateway üzerinden yönetilmektedir. Her mikroservis bağımsız çalışacak şekilde konumlandırılmış ve veri izolasyonu sağlanmıştır.

### *A. Richardson Olgunluk Modeli ve RESTful İletişim*

Sistem, REST mimari prensiplerine uygun olarak stateless (durumsuz) bir yapıda geliştirilmiştir. Servisler arası iletişim HTTP protokolü üzerinden sağlanmakta ve veri alışverişi JSON formatında gerçekleştirilmektedir.

API tasarımında Richardson Olgunluk Modeli Seviye 2 esas alınmıştır. Bu kapsamda her varlık için ayrı URI tanımlamaları yapılmış (örneğin /products, /orders, /auth) ve işlemler uygun HTTP metodları (GET, POST, PUT, DELETE) ile gerçekleştirilmiştir. Bu yaklaşım, sistemin standartlara uygun ve sürdürülebilir bir yapıya sahip olmasını sağlamaktadır.

### *B. Katmanlı Struct Yapısı ve Clean Architecture*

Tüm mikroservisler, Golang kullanılarak Clean Architecture prensiplerine uygun katmanlı bir yapı ile geliştirilmiştir. Bu yapı, veri akışını kontrol altına alırken sistemin modülerliğini artırmaktadır.

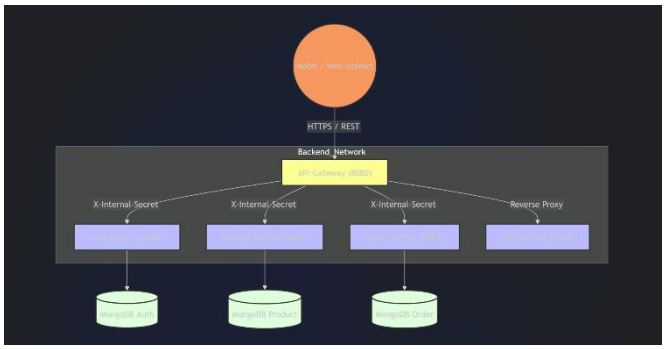
Model katmanı, MongoDB ile uyumlu veri yapılarını (BSON etiketleri ile) tanımlar. Repository katmanı, veri erişim işlemlerini gerçekleştirir ve MongoDB ile doğrudan iletişim kurar. Bu katmanda indeksleme kullanılarak veri erişim performansı optimize edilmiştir.

Service katmanı, iş mantığını barındırır ve veri doğrulama, hesaplama ve hata yönetimi işlemlerini gerçekleştirir. Handler katmanı ise HTTP isteklerini karşılar, JSON dönüşümlerini yapar ve uygun HTTP durum kodlarını üretir.

### *C. Genel Sistem Mimarisi*

Sistem, istemci ile mikroservisler arasındaki tüm iletişimi yöneten merkezi bir API Gateway üzerine kurulmuştur. API Gateway, gelen istekleri ilgili servislere yönlendirmenin yanı sıra kimlik doğrulama, yetkilendirme ve trafik kontrolü görevlerini üstlenmektedir.

Mikroservisler, güvenli bir ağ içerisinde konumlandırılmış olup dış dünyaya doğrudan açık değildir. Tüm erişimler yalnızca API Gateway üzerinden sağlanmaktadır. Servisler arası iletişim ise özel güvenlik başlıkları ile kontrol edilmektedir.




Şekil 1: API Gateway tabanlı mikroservis mimarisinin genel görünümü

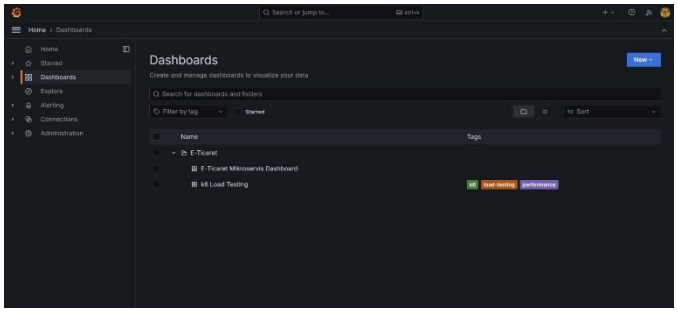
Her mikroservis belirli bir iş alanına odaklanmakta ve kendi veri tabanına sahiptir. Bu sayede bir serviste oluşabilecek hata diğer servisleri doğrudan etkilememektedir.

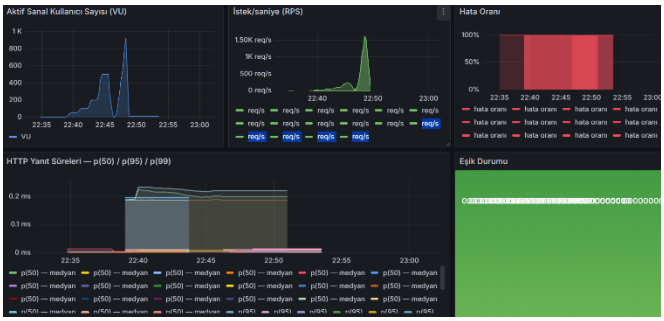
### *D. Sistem Testleri ve Performans Analizi*

Sistemin doğruluğu ve performansı k6 aracı ile test edilmiştir. Farklı yük senaryoları altında sistemin yanıt süreleri ve hata oranları analiz edilmiştir.

Düşük yük altında gerçekleştirilen "Smoke Test" senaryosunda sistemin tüm bileşenlerinin doğru çalıştığı doğrulanmış ve %100 başarı oranı elde edilmiştir. API Gateway'in istekleri doğru şekilde yönlendirdiği gözlemlenmiştir.

Yüksek trafik senaryosunda ise 100 eş zamanlı kullanıcı ile stres testi uygulanmıştır. Bu test sonucunda API Gateway üzerinde bulunan rate limiting mekanizması devreye girmiş ve kapasite aşımı durumlarında HTTP 429 hata kodu döndürülerek sistemin aşırı yüklenmesi engellenmiştir.






Şekil 2: k6 ile gerçekleştirilen yük testi sonuçları

### E. *Sonuçlar, Sınırlılıklar ve Geliştirme Önerileri*

Geliştirilen sistem, mikroservis mimarisinin sağladığı ölçeklenebilirlik ve esneklik avantajlarını başarılı bir şekilde sunmaktadır. Servislerin bağımsız olarak ölçeklenebilmesi sayesinde yoğunluk durumlarında yalnızca ilgili servis genişletilerek performans artırılabilmektedir.

Bununla birlikte, dağıtık yapı bazı zorlukları da beraberinde getirmektedir. Servislerin ayrı veri tabanlarına sahip olması nedeniyle karmaşık veri ilişkileri doğrudan yönetilememekte ve bazı işlemler için birden fazla servis çağrısı yapılması gerekmektedir.

Ayrıca hata ayıklama süreçleri daha karmaşık hale gelmiş olup sistem genelinde izlenebilirlik ihtiyacı ortaya çıkmıştır. Gelecek çalışmalarda merkezi loglama ve dağıtık izleme mekanizmalarının entegrasyonu planlanmaktadır.

## IV. API GATEWAY MİKROSERVİSİ

E-ticaret platformumuzun dış çeperi ve ağ güvenliğinin merkezi API Gateway'dir. Geleneksel yapılarda kullanıcılar doğrudan servislere erişebilirken, bu mimaride tüm IP tabloları kapatılmıştır ve yalnızca 8080 portunu dinleyen API Gateway aracılığıyla trafik geçişine izin verilmektedir. Bu yaklaşım, sistemin güvenlik izolasyonunu güçlendirmekte ve servisler arasındaki iletişimi tek bir kontrol noktasından yönetmemizi sağlamaktadır.

### A. *Struct Yapıları ve Algoritmalar*

API Gateway içerisinde temel veri yapıları, proxy loglarını ve rate-limit mekanizmalarını yönetir. Örnek olarak rate limiting algoritması, bellek-içi bir map[string]UserTracker kullanarak her kullanıcı isteğini O(1) sürede izler. Bu yaklaşım sayesinde gelen her IP veya token, hash tablosu üzerinden anında kontrol edilir. Eğer bir kullanıcı limiti aşarsa (ör. 60 istek/dk), sistem veritabanına gitmeden HTTP 429 Too Many Requests cevabını üretir.

Bellekte tutulan loglar düzenli olarak çöp toplayıcı tarafından temizlendiği için, bellek tüketimi O(N) lineer bir karmaşıklıkla yönetilmektedir. Bu sayede sistem, yüksek trafikte dahi performans kaybı yaşamaz.

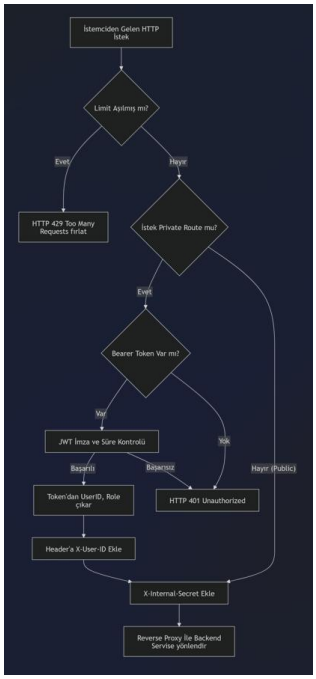
### B. *Akış ve İşleyiş*

API Gateway'in temel işleyişi, JWT doğrulaması, rate limiting ve ağ izolasyonu etrafında şekillenir. Gelen istekler önce limit kontrolünden geçer; limit aşıldıysa kullanıcıya hemen 429 döner. Limit aşılmadıysa istek private route ise token doğrulaması yapılır; public route ise arka servise özel X-Internal-Secret header'ı eklenir.

JWT doğrulaması sırasında token geçerli ise kullanıcı bilgileri çıkarılır ve header'a eklenir. Böylece arka servislere giden istekler, yetkisiz erişime karşı güvenle korunur.

### C. *Test Senaryoları*

API Gateway üzerinde yapılan yük testlerinde, saniyede binlerce isteğe rağmen latency 5ms'nin altında kalmıştır. Örneğin, geçersiz token ile 50.000 sanal kullanıcı isteği gönderildiğinde, Gateway arka servisleri hiç yüklememiş ve hepsine HTTP 401 dönmüştür. Bu sonuç, sistemin hem güvenliğini hem de performansını açık şekilde göstermektedir.




## V. AUTHENTİCATİON SERVİSİ

### A. *Struct Yapıları ve Karmaşıklık*

Auth Service'de kullanıcı bilgilerini temsil eden temel veri yapısı User struct'ıdır:

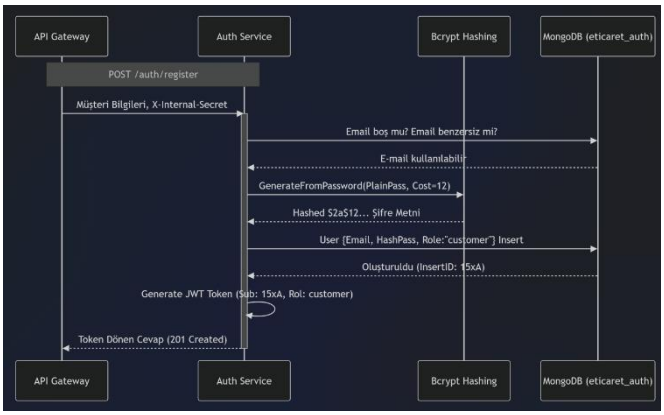
```
type User struct {
  ID       string `bson:"_id,omitempty"`
  Email    string `bson:"email"`
  Password string `bson:"password"`
  FirstName string `bson:"first_name"`
  LastName  string `bson:"last_name"`
  Role     string `bson:"role"`
}
```

Parolalar Bcrypt ile Cost=12 değeri kullanılarak hashlenir. Bu, brute-force veya rainbow table saldırılarına karşı sistemi üssel olarak korur; şifre çözme süresi yüksek işlem yükü gerektirir. Kullanıcı e-posta alanı MongoDB'de unique index ile tutulur, böylece mükerrer kayıt sorguları O(log N) karmaşıklıkla hızlıca doğrulanır.

### B. *Auth Akışı ve Sequence Diyagramı*

- Kullanıcı kayıt ve token alma süreci aşağıdaki adımları içerir:

- API Gateway üzerinden /auth/register endpoint'ine POST isteği gönderilir.
- Auth Service, email'in benzersizliğini MongoDB üzerinden kontrol eder.
- Şifre Bcrypt ile hashlenir (Cost=12).
- Kullanıcı veritabanına eklenir.
- JWT token üretilir ve API Gateway'e geri döner.




3. Testler ve Performans

Bcrypt işlemleri CPU üzerinde yoğun yük oluşturur. Yük testlerinde (load-tests/), saniyede yüksek sayıda login isteği atıldığında CPU sınırlarına yaklaşmıştır. Özellikle Timing Attack önlemleri uygulanmıştır: hatalı şifre girişleri bile her zaman aynı süre içinde yanıt verir. Bu, saldırganların cevap süresinden şifre doğruluğunu tahmin etmesini engeller.

## VI. Ürün Servisi

E-Ticaret sistemlerinde trafiğin büyük bir kısmını Ürün Okuma (Read-Heavy) işlemleri oluşturur. Product Service, kataloglanmış ürünlerin depolandığı, arandığı ve listelendiği merkezi bir servis olarak tasarlanmıştır. Bu yapı sayesinde ürün stokları ve detayları, diğer servislerin iş yükünden bağımsız bir şekilde yönetilebilir.

### *A. 1. Modelleme ve Karmaşıklık Analizi*

Ürün verilerini temsil eden temel struct aşağıdaki gibidir:

```
type Product struct {
  ID        string   `bson:"_id,omitempty"`
  Title     string   `bson:"title"`
  Description string  `bson:"description"`
  Price     float64  `bson:"price"`
  Stock     int      `bson:"stock"`
  CategoryIDs []string `bson:"category_ids"`
  IsActive  bool     `bson:"is_active"`
}
```

#### *1) Zaman ve Mekan Karmaşıklığı:*

Ürün arama ve filtreleme işlemleri MongoDB üzerinde B-Tree indexleri kullanılarak O(log N) seviyesinde çalışır. Bu, kategori ID veya başlık bazlı aramalarda performansı artırır.

Servis tamamen okuma odaklı olduğundan, milyonlarca kullanıcı aynı anda ürün listesini sorgulasa bile deadlock veya lock-waiting gibi I/O problemleri yaşanmaz.

Bellek kullanımı, yalnızca isteğin RAM'de kapladığı alan ile sınırlıdır (O(1)), bu sayede servis hafif ve verimli kalır.

### *B. Akış ve Sistem Mimarisi*

Ürün servisi, anonim kullanıcıların API Gateway üzerinden geçerek ürün listesini almasını sağlar. Gateway, X-Internal-Secret header'ı ile güvenlik ekler. Servisin iç mimarisi ise handler → servis mantığı → repository → MongoDB şeklinde katmanlıdır. Bu diyagram, ürün listesinin API Gateway'den geçerek MongoDB'den çekilmesini ve kullanıcıya ulaştırılmasını açıkça göstermektedir.

## VII. Sipariş Servisi

Order Service, e-ticaret sisteminde Write-Heavy (Yazma Yoğun) işlemlerin merkezi kapısıdır. Kullanıcının sepetini onaylayıp siparişe dönüştürmesini sağlar ve parasal işlemlerin eşzamanlılığını garanti altına alır. Servis, sistemin en kritik iş parçacıklarını barındırır ve veri bütünlüğünü korumaktan sorumludur.

### *A. Struct Tasarımı ve Servis İletişimi*

Siparişleri temsil eden temel yapı aşağıdaki gibidir:

```
type Order struct {
  ID        string    `bson:"_id,omitempty"`
  UserID    string    `bson:"user_id"`
  Products  []OrderItem `bson:"items"`
  TotalAmount float64  `bson:"total_amount"`
  Status    string    `bson:"status"` // pending, approved, shipped
  CreatedAt time.Time `bson:"created_at"`
}
```

- Products alanı, siparişin içindeki her kalemi OrderItem struct'ı ile listeler.
- TotalAmount sunucuda hesaplanır; istemcinin gönderdiği fiyat verisine güven yoktur.
- Status, siparişin geçerlilik durumunu takip eder (pending, approved, shipped).

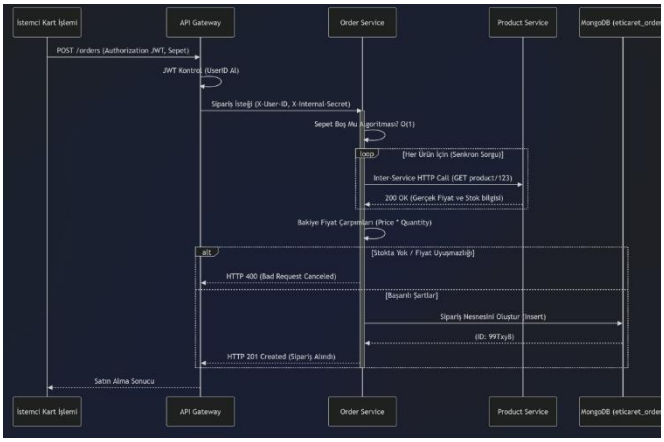
Senkron Servis İletişimi:

Order Service, ürün fiyatlarını ve stok durumlarını almak için Product Service'e HTTP üzerinden senkron olarak bağlanır. Örneğin sepette 10 ürün varsa, her bir ürün için ayrı bir HTTP GET çağrısı yapılır. Paralel işleme kullanılmazsa zaman karmaşıklığı O(M) olur ve gecikmeler oluşabilir.

*B. Sipariş Algoritması ve Sequence Diyagramı*

Müşterinin token ile sipariş oluşturma süreci aşağıdaki gibi işlenir:

Diyagram Açıklaması:




- API Gateway, JWT doğrulamasını yapar ve X-User-ID ile siparişi Order Service'e yönlendirir.
- Order Service her ürün için Product Service'e senkron çağrı yapar, stok ve fiyat bilgilerini doğrular.
- Tüm doğrulamalar tamamlandıysa sipariş MongoDB'ye kaydedilir ve kullanıcıya başarı mesajı döner.

3. Uygulama Detayları ve İş Süreci

Kullanıcının gönderdiği sepet fiyatlarına güvenilmez; sistem sunucuda fiyatları ve stokları doğrular.

K6 yük testlerinde ardışık yüzlerce sipariş eklenmiş, OrderDB herhangi bir blokaj veya gecikme yaşamadan çalışmıştır.

Sistem, parasal veri bütünlüğünü sağlamak için kritik bir iş mantığı katmanı içerir.

## VIII. CDN Servisi

CDN Service, projenin statik içerik dağıtım merkezi olarak görev yapar. Dinamik veri üretimi gerektirmeyen, sadece okunabilir statik görsellerin (ürün resimleri, banner'lar vb.) kullanıcı tarayıcılarına iletilmesini sağlar.

Bu servis, diğer mikroservislerden bağımsız çalışır ve yüksek trafik altında dahi performans kaybı yaşamayacak şekilde optimize edilmiştir.

*A. Teorik Altyapı ve Dosya Yönetimi*

cdn-service, klasik bir HTTP yığını değil, statik FileServer gibi çalışır. Karmaşık veritabanı sorguları veya kimlik doğrulama mekanizmalarına ihtiyaç duymaz.

*1) Zaman ve Mekan Karmaşıklığı:*

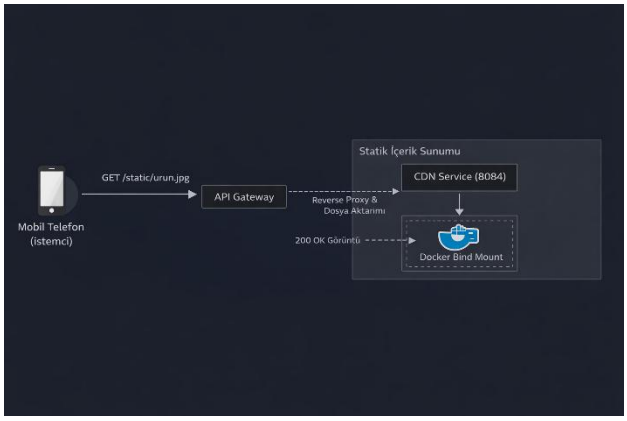
Zaman Karmaşıklığı (Time Complexity): Dosya sunumu disk I/O ve seek sürelerine bağlıdır, request başına O(1)'dir.

Mekan Karmaşıklığı (Space Complexity): Statik içerikler chunk bazlı okunarak gönderilir; RAM üzerinde sadece geçici buffer tutulur, O(1) seviyesindedir.

Bu yaklaşım, milyonlarca görsel talebinde bile sistemin minimal bellek ve CPU kullanmasını sağlar.

*B. Katman ve Veri Akış Diyagramı*

Statik bir resim istemi geldiğinde veri akışı aşağıdaki gibi işler:




Diyagram Açıklaması:

- Kullanıcı isteği API Gateway üzerinden CDN Service'e yönlendirilir.
- CDN Service, Docker bind mount ile bağlı host filesystem'den statik dosyayı okur.
- Dosya chunk'lar halinde stream edilerek istemciye geri döner.
- Eğer dosya yoksa 404 Not Found cevabı verilir.

Bu akış, backend servislerin yükünü tamamen bypass eder ve statik içeriklerin hızlı iletimini sağlar.

3. Uygulama Detayları

Ürünlerin görsel verileri (iphone15.jpg, airpodspro.jpg) fiziksel dosya olarak saklanır; veritabanına Base64 ile gömülmez.

Mobil veya web uygulama, image_url üzerinden CDN sunucusuna asenkron olarak yönlenir.

HTTP status kodları üzerinden görüntü durumu kontrol edilir, eksik görsel varsa kullanıcıya kırık imaj ikonu gösterilir.

Bu yaklaşım, veritabanı şişkinliği ve network tıkanıklığını önler.

## Kaynaklar


[1] Smith, J. O. and Abel, J. S., ``Bark and ERB Bilinear Transforms", *IEEE Trans. Speech and Audio Proc.*, 7(6):697-708, 1999.

[2] Lee, K.-F., *Automatic Speech Recognition: The Development of the SPHINX SYSTEM,* Kluwer Academic Publishers, Boston, 1989.

[3] Rudnicky, A. I., Polifroni, Thayer, E H., and Brennan, R. A. "Interactive problem solving with speech", *J. Acoust. Soc. Amer., Vol. 84, 1988, p S213(A).*